

Solution for CS5275 Problem Set 3

Prepared by TA Ivan Adrian Koswara

Contents

1 Problem 1	2
1.1 My solution	2
1.1.1 Generalizing the problem	2
1.1.2 Tutorial problem set 2 problem 2	2
1.1.3 The actual proof	4
1.2 Insight and discussion	5
1.3 Grading criteria and common mistakes	5
1.3.1 Minor error: Not explaining why later steps are also $\leq \phi$	5
1.4 Appendix: Stationary distribution on G , not on S	5
2 Problem 2	7
2.1 My solution	7
2.2 Insight and discussion	8
2.3 Grading criteria and common mistakes	8
2.4 Appendix: Better (b)	8
3 Problem 3	9
3.1 My solution	9
3.2 Insight and discussion	9
3.3 Grading criteria and common mistakes	10
3.3.1 Major error: Not spotting the small $ L $ case	10
3.4 Appendix: Upper-bounding $ E(L, C) $ another way	10
4 Problem 4	11
4.1 My solution	11
4.1.1 The algorithm and its complexity	11
4.1.2 Step 1 returns a cut	12
4.1.3 Will find small min-cuts	12
4.1.4 The algorithm is correct	12
4.2 Insight and discussion	13
4.2.1 Proving the correctness	13
4.2.2 Deriving the algorithm given G'	13
4.2.3 Getting the idea of r	13
4.3 Grading criteria and common mistakes	14
4.3.1 Minor error: Not proving the algorithm will not hallucinate	14

1 Problem 1

Let G be a graph and let $(S, V \setminus S)$ be a cut with conductance $\Phi(S) = \phi$. We consider a standard (non-lazy) random walk, starting from any $v \in S$, for $t = \lfloor 0.1/\phi \rfloor$ steps.

Let X be the set of all $v \in S$ such that

$$\Pr(\text{the walk starting from } v \text{ never leaves } S) \geq \frac{1}{2}.$$

Prove

$$\text{vol}(X) \geq 0.8 \cdot \text{vol}(S).$$

1.1 My solution

1.1.1 Generalizing the problem

We will generalize the problem statement as follows. The changes are marked in red.

Let G be a graph and let $(S, V \setminus S)$ be a cut with conductance $\Phi(S) = \phi$. We consider a standard (non-lazy) random walk, starting from any $v \in S$, for $t = \lfloor \tau/\phi \rfloor$ steps, **for some constant $\tau > 0$.**

Fix a constant δ . Let $S \setminus X$ be the set of all $v \in S$ such that

$$\Pr(\text{the walk starting from } v \text{ leaves } S \text{ at some point}) \geq \delta.$$

Let **μ be a constant** satisfying

$$\delta \cdot \mu \geq \tau.$$

Prove that

$$\text{vol}(S \setminus X) \leq \mu \cdot \text{vol}(S).$$

We detail these changes:

- We generalize the constants $0.1, \frac{1}{2}, 0.8$ in the problem into $\tau, 1 - \delta, 1 - \mu$ respectively. (See below for why we use $1 - \delta$ and $1 - \mu$.)
- It follows that for the problem statement, we have $\tau = 0.1, \delta = 0.5, \mu = 0.2$. (If you're curious, our draft of the problem had $\tau = 0.001, \delta = 0.5, \mu = 0.5$. We know this is true for small enough τ ; we simply tightened up the bounds.)
- We also focus on the walks that *do* leave S , instead of the walks that don't. As such, we look at $S \setminus X$ rather than X , both for the definition and for the final volume inequality. This explains the $1 - \delta$ and $1 - \mu$ used for the original problem statement.

1.1.2 Tutorial problem set 2 problem 2

To prove this generalized problem statement, we will make use of Tutorial problem set 2 problem 2. However, as there is no published official solution for that problem, we will have to prove anything we want to use. The intuition is as described by Prof: "kill off" probability mass that escapes S . However, I found this particularly difficult to formalize. This is my attempt at formalizing it; yours might differ.

We first modify G into a directed multigraph G^* as follows. The motivation follows below.

- Contract everything in $V \setminus S$ into a single vertex v^* . This contraction should **preserve all edges in the cut**, possibly resulting in multi-edges with multiplicity. For example, if $s \in S$ is adjacent to two vertices $x, y \in V \setminus S$, then now there are two edges between s and v^* .
- For every (undirected) edge inside S , split it into two directed edges in opposite directions.
- For every edge in the cut $E(S, v^*)$, direct it into v^* .
- Add a self-loop on v^* .
- The random walk becomes directed: it will only go through outgoing edges.

I will now explain what this modification is doing.

The direction of the cut is my formalization for “killing off” the probability mass. Once a mass leaves S , it cannot return back to S . Simply erasing the mass is mathematically not quite rigorous (where does the probability go?), so instead I modify the graph so the mass still exists, but does not return to S .

Also, we don’t care about what happens to the mass once it leaves S . So I decided to contract the entire $V \setminus S$ into a single vertex with a self-loop, to represent “the mass is somewhere in $V \setminus S$ and won’t leave it”. We need to preserve the number of edges in the cut, so I emphasize that the edges remain with multiplicity.

Finally, because we have directed edges in the cut, it’s easier to argue when the entire graph is directed. That’s why I also split edges in S into a pair of directed arcs. To the random walk, as long as it has not left S , the graph looks identical to G . Once it leaves S , we no longer care what it does, so we trap it outside S .

Now, let $\mathbf{x}$ be a probability distribution on $S \cup \{v^*\}$, i.e. the vertex set of G^* . We state and prove the following lemma.

Lemma 1.1. *Suppose $x_v \leq \deg(v)/\text{vol}(S)$ for all $v \in S$. Suppose that after one step of the random walk, we obtain the probability distribution $\mathbf{x}'$. Then:*

- (a) $x'_v \leq \deg(v)/\text{vol}(S)$ for all $v \in S$.
- (b) $x'_{v^*} - x_{v^*} \leq \phi$.

Here $\deg(v)$ means the outer-degree of v , which is equal to $\deg_S(v)$ in the induced subgraph $G[S]$ of the original graph. In a directed graph, we use the convention that $\text{vol}(S)$ counts each edge outgoing from some vertex in S , so it counts the same thing as in the original G .

We quickly explain the intuition behind the lemma. The premise says, the probability distribution $\mathbf{x}$ is already “quite distributed” on S , being upper-bounded by the stationary distribution on S . Item (a) of the lemma says that the resulting probability distribution will still be quite distributed in the same way; we will use this fact to apply the lemma repeatedly in an induction. Item (b) is the main result: the probability mass that goes to v^* increases by at most ϕ each step.

Proof of Lemma 1.1. By definition of a random walk, we have the following for any vertex $v \in S \cup \{v^*\}$:

$$x'_v = \sum_{\text{edge } (u \rightarrow v)} \frac{x_u}{\deg(u)}$$

We divide into two cases.

Case 1: $v \in S$. Then every u that counts in the summation is also in S . By the premise, $x_u \leq \deg(u)/\text{vol}(S)$, therefore it contributes $\leq 1/\text{vol}(S)$ into the sum. The number of u ’s that count to the summation is exactly $\deg(v)$. Therefore

$$x'_v \leq \deg(v) \cdot \frac{1}{\text{vol}(S)} = \frac{\deg(v)}{\text{vol}(S)}.$$

Applying this to every $v \in S$, we obtain item (a).

Case 2: $v = v^*$. Recall that we count edges with multiplicity. If there are multiple edges $(u \rightarrow v)$, it contributes that many times in the sum.

One of the u that counts is $u = v^*$, which has $\deg(v^*) = 1$, so it sends the entire probability mass back to v^* . Every other u is in S , so by a similar analysis as above, it contributes $\leq 1/\text{vol}(S)$ into the sum. The number of u 's that count is exactly $|E(S, \{v^*\})| = |E(S, V \setminus S)|$. Therefore

$$x'_{v^*} - x_{v^*} \leq |E(S, V \setminus S)| \cdot \frac{1}{\text{vol}(S)} = \phi.$$

This gives item (b). □

1.1.3 The actual proof

We are finally ready to prove the generalized problem statement. Begin with the probability distribution

$$x_v = \frac{\deg(v)}{\text{vol}(S)} \text{ for all } v \in S, \text{ and } x_{v^*} = 0.$$

Take the random walk for $t = \lfloor \tau/\phi \rfloor$ steps, and suppose the final probability distribution is $\mathbf{x}'$. By applying Lemma 1.1 repeatedly and noting that $x_{v^*} = 0$,

$$x'_{v^*} - x_{v^*} \leq t \cdot \phi \quad \implies \quad x'_{v^*} \leq \tau$$

Now consider random walks for t steps. We call a random walk to “escape” if it leaves S (and hence is stuck in v^*) within t steps. Define the vector $\mathbf{p}$ on vertices in S , such that for $v \in S$, p_v is the probability a random walk starting from v escapes. Then the probability that a random walk escapes, starting from the probability distribution $\mathbf{x}$, is simply the weighted average

$$\mathbf{x} \cdot \mathbf{p} = \sum_{v \in S} x_v p_v.$$

However, any random walk that escapes must end on v^* . Therefore the probability that a random walk escapes is simply x'_{v^*} . Therefore we have the inequality

$$\sum_{v \in S} x_v p_v = x'_{v^*} \leq \tau$$

Now let $S \setminus X$ be the set of vertices v for which $p_v > \delta$. By keeping only the vertices in $S \setminus X$ and using the bound on p_v , we get

$$\tau \geq \sum_{v \in S} x_v p_v \geq \sum_{v \in S \setminus X} x_v \delta \quad \implies \quad \sum_{v \in S \setminus X} x_v \leq \frac{\tau}{\delta} \leq \mu$$

However,

$$\sum_{v \in S \setminus X} x_v = \sum_{v \in S \setminus X} \frac{\deg(v)}{\text{vol}(S)} = \frac{\text{vol}(S \setminus X)}{\text{vol}(S)}$$

Therefore,

$$\frac{\text{vol}(S \setminus X)}{\text{vol}(S)} \leq \mu,$$

proving the generalized problem statement.

1.2 Insight and discussion

The solution looks long, but the intuition is relatively simple. I just found it surprisingly difficult to formalize the intuition. (And there's also an early discussion about generalizing the problem statement.)

Core to the solution is the hint: tutorial problem set 2 problem 2, about showing that only $\leq \phi$ probability mass can escape S every step. The main problem is formalizing it, and Section 1.1.2 (Lemma 1.1) is fully devoted for that.

Once you have the result, it's pretty easy to wrap up the solution. Consider the "bad vertices": the vertices for which the probability is high enough to escape S . Then the weighted average $\mathbf{x} \cdot \mathbf{p}$ can be interpreted as two ways: the probability that we pick a bad vertex, and also the probability that a random walk escapes S . Not just that: given that we start with $\mathbf{x}$ being the stationary distribution, our desired number $\text{vol}(S \setminus X)/\text{vol}(S)$ is also exactly the same probability. This observation carries the end of the solution.

The problem has several degrees of freedom regarding the choices of constants, and I decided to generalize all those constants. Originally, we were looking for 0.5 of the volume (rather than 0.8), with a time bound of $0.001/\phi$ (rather than $0.1/\phi$). We decided to tighten up the bounds because the calculations are the same either way.

1.3 Grading criteria and common mistakes

1.3.1 Minor error: Not explaining why later steps are also $\leq \phi$

This is by far the most common error. If you start with $\mathbf{x}$ where $x_v = \text{deg}(v)/\text{vol}(S)$, then it's easy to prove that the probability of escaping S is $\leq \phi$ for the first step, by using the formula derived from random walks,

$$x'_v = \sum_{\text{edge } uv} \frac{x_u}{\text{deg}(u)}$$

The problem is, when you prove your statement, you're using $x_u = \text{deg}(u)/\text{vol}(S)$. With an equality sign. After the first step, this equality no longer holds. The probability distribution $\mathbf{x}$ has changed across the graph. Therefore, your proof also no longer holds beyond the first step. This is why, in Lemma 1.1, I had to include item (a): to show that the premise still holds, so I can use the lemma again for the next step.

If you don't argue anything about this, it's a minor error that docks 1 point.

1.4 Appendix: Stationary distribution on G , not on S

One alternative solution is to consider the stationary distribution on the whole G , rather than just on S .

Let $\mathbf{x}$ be the stationary distribution on G , i.e.

$$x_v = \frac{\text{deg}(v)}{\text{vol}(G)}.$$

Now, the probability of transitioning through any particular edge (in a specific direction) is always $1/\text{vol}(G)$. Moreover, because $\mathbf{x}$ is stationary, after one step of the random walk, the probability distribution is $\mathbf{x}$ again. This bypasses the error mentioned in Section 1.3.1.

The probability that, at any given step, a random walk moves from S to $V \setminus S$ is simply

$$\frac{|E(S, V \setminus S)|}{\text{vol}(G)}.$$

Now, condition this probability on the event that the random walk is currently in S . This happens with probability $\text{vol}(S)/\text{vol}(G)$. Thus, given that the random walk is currently in S , the probability it moves to $V \setminus S$ is

$$\frac{|E(S, V \setminus S)|}{\text{vol}(G)} \bigg/ \frac{\text{vol}(S)}{\text{vol}(G)} = \frac{|E(S, V \setminus S)|}{\text{vol}(S)} = \phi$$

The difference with the official solution is, this result holds for any step in the random walk, not just the first step.

Finally, wrap up with the union bound. The probability a walk that starts from S ever escapes within t steps is

$$\begin{aligned}\Pr[\text{escapes in } t \text{ steps}] &= \sum_{i=1}^t \Pr[\text{escapes on the } i\text{-th step}] \\ &= \sum_{i=1}^t \Pr[\text{stays in } S \text{ for } i-1 \text{ steps}] \cdot \Pr[\text{moves from } S \text{ to } V \setminus S] \\ &\leq \sum_{i=1}^t (1 \cdot \phi) \\ &= t \cdot \phi\end{aligned}$$

2 Problem 2

Let G be any Ramanujan graph that has n vertices and is d -regular.

- (a) Prove that for any cut $(S, V \setminus S)$,

$$|E(S, V \setminus S)| \leq \frac{nd}{4} + O(n\sqrt{d}).$$

- (b) Prove that there exists a cut $(S, V \setminus S)$ such that

$$|E(S, V \setminus S)| \geq \frac{nd}{4} - O(n\sqrt{d}).$$

2.1 My solution

The key idea is the expander mixing lemma (lecture 8 slide 6):

$$\left| e(S, T) - \frac{d}{n} \cdot |S| \cdot |T| \right| \leq \sigma_2 \cdot \sqrt{|S| \cdot |T|}$$

When $T = V \setminus S$, the number $e(S, T) = |E(S, V \setminus S)|$ gives the size of the cut.

Also, for a Ramanujan graph, $\sigma_2 \leq 2\sqrt{d-1}$.

In addition to the expander mixing lemma, we make use of the following inequality:

$$|S| \cdot |V \setminus S| \leq \frac{n^2}{4}$$

This is easily derived in many ways (such as by the AM-GM inequality), by observing $|S| + |V \setminus S| = n$.

- (a) Take the upper bound from the expander mixing lemma:

$$\begin{aligned} |E(S, V \setminus S)| &\leq \frac{d}{n} \cdot |S| \cdot |V \setminus S| + \sigma_2 \cdot \sqrt{|S| \cdot |V \setminus S|} \\ &\leq \frac{d}{n} \cdot \frac{n^2}{4} + 2\sqrt{d-1} \cdot \sqrt{\frac{n^2}{4}} \\ &= \frac{nd}{4} + n\sqrt{d-1} \\ &= \frac{nd}{4} + O(n\sqrt{d}) \end{aligned}$$

- (b) For the following, we assume n is even. (The case n is odd is left as an exercise; it just changes $O(1)$ to the number below.)

Take any $|S| = n/2$. Then $|V \setminus S| = n/2$ too, so $|S| \cdot |V \setminus S| = n^2/4$; specifically, it is an equality rather than an upper bound.

Take the lower bound from the expander mixing lemma:

$$\begin{aligned} |E(S, V \setminus S)| &\geq \frac{d}{n} \cdot |S| \cdot |V \setminus S| - \sigma_2 \cdot \sqrt{|S| \cdot |V \setminus S|} \\ &= \frac{d}{n} \cdot \frac{n^2}{4} - \sigma_2 \cdot \sqrt{\frac{n^2}{4}} \\ &\geq \frac{d}{n} \cdot \frac{n^2}{4} - 2\sqrt{d-1} \cdot \sqrt{\frac{n^2}{4}} \\ &= \frac{nd}{4} - n\sqrt{d-1} \\ &= \frac{nd}{4} - O(n\sqrt{d}) \end{aligned}$$

2.2 Insight and discussion

This proof is incredibly simple once you know what you want: the expander mixing lemma. This lemma is applicable for all graphs, but is especially powerful for Ramanujan graphs (whose σ_2 is small). The rest is simply making the observation that you want $T = V \setminus S$, and that you have the bound $|S| \cdot |T| \leq n^2/4$ and that it can be achieved (or nearly achieved for odd n).

2.3 Grading criteria and common mistakes

I believe everyone did this problem correctly, well done.

2.4 Appendix: Better (b)

As a lot of you noticed, the statement for (b) is incredibly weak. There's a simpler way to obtain a better bound, which also applies to all graphs.

We use probabilistic method. Construct S randomly, by choosing whether each vertex is included in S with probability $1/2$. Then every edge is in the cut $(S, V \setminus S)$ with probability exactly $1/2$. Therefore

$$\mathbb{E}[|E(S, V \setminus S)|] = \frac{|E|}{2}.$$

Therefore there exists some cut $(S, V \setminus S)$ with size $\geq |E|/2$. In a d -regular graph, we have $|E| = nd/2$. This gives a cut of size $\geq nd/4$, even ignoring the $-O(n\sqrt{d})$ term.

I'm not sure how we missed this solution; it should be extremely straightforward. But there's an interesting observation for Ramanujan graphs, derived from the main solution.

Note that *any* S with $|S| = n/2$ will give a cut of the desired size. So the approach using expander mixing lemma is constructive (just pick any S with $|S| = n/2$), and it also proves there are at least $\binom{n}{n/2} = O(2^n/\sqrt{n})$ such cuts, exponentially many of them. The probabilistic method is inherently non-constructive (even if [it can be derandomized](#)), and can only give 1 cut.

3 Problem 3

Let G be a connected d -regular graph with conductance $\Phi(G) \geq \phi$.

Prove that the vertex connectivity $\kappa(G)$ satisfies

$$\kappa(G) = \Omega(\phi \cdot d).$$

3.1 My solution

Here we assume G is not a complete graph, so there is a vertex cut. (If G is a complete graph, then $\phi = \Theta(1)$, and $\kappa(G) = d = n - 1$, so the statement is true anyway.)

Let (L, C, R) be an arbitrary vertex cut, and without loss of generality, suppose $\text{vol}(L) \leq \text{vol}(R)$. We claim $|C| = \Omega(\phi \cdot d)$. If we prove this is true for all vertex cuts, it also follows $\kappa(G) = \min |C| = \Omega(\phi \cdot d)$.

First, note that because (L, C, R) is a vertex cut, it follows there are no edges between L and R . Therefore $E(L, V \setminus L) = E(L, C)$. We will investigate this cut.

There are two cases depending on $|L|$.

Case 1: $|L| \geq d/2$. Then

$$|E(L, V \setminus L)| \geq \phi \cdot \text{vol}(L) \geq \phi \cdot \left(d \cdot \frac{d}{2}\right) = \Omega(\phi \cdot d^2)$$

However,

$$|E(L, V \setminus L)| = |E(L, C)| \leq d \cdot |C|$$

Combining the two inequalities, we obtain

$$d \cdot |C| = \Omega(\phi \cdot d^2) \implies |C| = \Omega(\phi \cdot d)$$

Case 2: $|L| < d/2$. Take any vertex $v \in L$. Because the graph is simple, v must have d distinct neighbors. There can only be $\leq d/2$ of them in L , so there are $\geq d/2$ neighbors outside L . But $E(L, V \setminus L) = E(L, C)$, so all edges going out of L point to C . So v has $\geq d/2$ neighbors in C .

Therefore $|C| \geq d/2$. Since $\phi \leq 1$, we have

$$|C| = \Omega(d) = \Omega(\phi \cdot d)$$

So we have proven $|C| = \Omega(\phi \cdot d)$ for all vertex cuts. The conclusion follows.

3.2 Insight and discussion

The main idea here is really simple: if (L, C, R) is a vertex cut, then there are no edges between L and R , all edges going out of L must go to C . This allows the conductance to give a lower bound on $|E(L, C)|$. You just need to derive an upper bound in another way, and then chain the inequalities together.

There are a few ways to derive a bound for $|E(L, C)|$, but all of them crucially rely on the fact that G is simple. In fact, if G is not simple, the statement is false: consider a cycle graph, except each edge has multiplicity $d/2$. You have $\kappa(G) = 2$, but you can bring d however large you want, and ϕ doesn't depend on d (although it depends on the number of vertices n).

My solution looks at the L side. It feels more natural, since you have $\text{vol}(L)$ on the conductance. If you do this though, you will have to divide your solution into two cases: if L is large enough (so $|E(L, C)|$ has a ton of edges), and if L is small (and you use the simple-ness).

An alternative solution, which was submitted by many of you, is to count $|E(L, C)| \leq |L| \cdot |C|$ and work from there. You can read it in Section 3.4.

3.3 Grading criteria and common mistakes

3.3.1 Major error: Not spotting the small $|L|$ case

If you did not specifically handle the small $|L|$ case, you can only conclude that $\text{vol}(L) = d \cdot |L|$, and you end up with the conclusion

$$|S| \geq \phi \cdot |L|.$$

This does not solve the problem. Also, you have not made use of the fact that G is simple. As described in the discussion, the problem is false if G is not necessarily simple.

3.4 Appendix: Upper-bounding $|E(L, C)|$ another way

There is a simpler way to obtain the inequality, which actually gives a better constant in the Ω (1 instead of $1/2$). We count $|E(L, C)|$ in two ways.

First, because the graph is simple,

$$|E(L, C)| \leq |L| \cdot |C|$$

Second, by using the conductance similar to the main solution,

$$|E(L, C)| = |E(L, V \setminus L)| \geq \phi \cdot \text{vol}(L) = \phi \cdot d \cdot |L|$$

Combining these two observations,

$$\phi \cdot d \cdot |L| \leq |L| \cdot |C| \quad \implies \quad |C| \geq d \cdot \phi$$

4 Problem 4

Suppose we have an oracle that computes an s - t minimum vertex cut.

Let G be a graph with n vertices, and let $k = \sqrt{n}$. Let S be an arbitrary set of $k+1$ vertices, and construct a new graph G' derived from G by adding one new vertex r that is adjacent to everything in S (and no other).

Design (and prove the correctness of) an algorithm with the following properties:

- It makes at most $O(n)$ oracle queries.
- Aside from the oracle queries, it takes time $O(n^2)$.
- It outputs one of the following:
 - A minimum vertex cut of G .
 - The statement “ $\kappa(G) > k$ ”.

4.1 My solution

In the following, “cut” always means a vertex cut (L, C, R) . Similarly, a “min-cut” means a minimum vertex cut (possibly only a minimum cut between two vertices, not necessarily globally). Recall that (L, C, R) is a cut iff both of the following are true:

- There is no edge between L and R .
- $L \neq \emptyset$ and $R \neq \emptyset$.

We assume that the oracle either gives an s - t min-cut, or states that s and t are equal/adjacent (in which there is no cut separating s and t). We also assume n is large enough so that G' is not a complete graph.

4.1.1 The algorithm and its complexity

Here’s the algorithm. In the following, we assume we simply skip querying the oracle whenever s and t are equal/adjacent.

1. For each $y \in V$, query the oracle for a min-cut between r and y in G' . Assume r is in the left side, otherwise flip the sides.
2. For each $x, y \in S$, query the oracle for a min-cut between x and y in G .
3. Among all the cuts obtained above, find one with the smallest cut size. Denote it (L, C, R) .
4. Return the output:
 - a) If $|C| > k$, return “ $\kappa(G) > k$ ”.
 - b) Otherwise, if the cut came from step 1, return $(L \setminus \{r\}, C, R)$.
 - c) Otherwise, the cut came from step 2. Simply return (L, C, R) .

We first compute its complexity.

Step 1 runs $O(n)$ query calls, and step 2 runs $O(k^2) = O(n)$ query calls. So in total we have $O(n)$ query calls.

To find the smallest cut size, we iterate over all $O(n)$ cuts we have. Assuming a reasonable encoding, we can find the cut size in $O(n)$ time (iterating over all vertices to find which ones are in C). So step 3 takes $O(n^2)$ time. The entirety of step 4 takes at most $O(n)$ time. So in total the algorithm does $O(n^2)$ time outside the oracle queries.

So the algorithm’s complexity is settled. It remains to prove the algorithm is correct. We will prove several claims, after which the correctness follows.

4.1.2 Step 1 returns a cut

Lemma 4.1. *If step 1 of the algorithm finds an (r, y) -cut (L, C, R) in G' with size $\leq k$, then $(L \setminus \{r\}, C, R)$ is a cut in G (with the same size).*

Proof. Suppose $r \in L$ and $y \in R$.

Since $|C| \leq k$ but $|S| = k + 1$, it follows there exists some vertex $s \in S \setminus C$. Since s is adjacent to r , it follows $s \in L$ too.

Now consider the cut (L', C, R) where $L' = L \setminus \{r\}$. Now $s \in L'$ so the left side is not empty, and clearly the right side R is not empty. Moreover, $E(L', R) \subseteq E(L, R) = \emptyset$, so there is no edge between L' and R . Therefore (L', C, R) is a cut in G . $\square$

Lemma 4.2. *If the algorithm returns a cut (L, C, R) of size $\leq k$, then $\kappa(G) \leq |C|$.*

Proof. If the algorithm finds one such cut from step 2, then it's clearly a cut in G , so $\kappa(G) \leq |C|$. Otherwise it's a cut from step 1, and by Lemma 4.1, its C is also a cut in G . $\square$

4.1.3 Will find small min-cuts

Lemma 4.3. *Consider a min-cut (in G) (L^*, C^*, R^*) , and suppose its size $\kappa(G)$ is $\leq k$. If there exists some $x \in L^*$ and $y \in R^*$ such that $x, y \in S$, then step 2 of the algorithm finds an (x, y) -cut (L, C, R) of size $\kappa(G)$.*

Proof. The oracle finds a min-cut between x and y . The cut (L^*, C^*, R^*) is clearly of minimum size, so the oracle will find a cut of equivalent size. $\square$

Lemma 4.4. *Consider a min-cut (in G) (L^*, C^*, R^*) , and suppose its size $\kappa(G)$ is $\leq k$. Also suppose $R^* \cap S = \emptyset$. Let $y \in R^*$ be an arbitrary vertex. Then step 1 of the algorithm finds an (r, y) -cut (L, C, R) (in G') of size $\kappa(G)$.*

Proof. By the assumption, $|C^*| \leq k$. However, $|S| = k + 1$, so there exists a vertex $x \in S$ that is not in C^* . Since the assumption also says $R^* \cap S = \emptyset$, it follows $x \in L^*$. Now we claim $(L^* \cup \{r\}, C^*, R^*)$ is a cut in G' .

Clearly both sides are not empty. Furthermore,

$$E(L^* \cup \{r\}, R^*) = E(L^*, R^*) \cup E(\{r\}, R^*).$$

However, $E(L^*, R^*)$ is empty because (L^*, C^*, R^*) is a cut in G . Also, $E(\{r\}, R^*)$ is empty because $R^* \cap S = \emptyset$, and r is only adjacent to vertices in S . Therefore $(L^* \cup \{r\}, C^*, R^*)$ is a cut in G' . It has size $|C^*| = \kappa(G)$.

The oracle finds a min-cut between r and y , so it will find a cut of equivalent size. $\square$

Lemma 4.5. *If $\kappa(G) \leq k$, the algorithm will return a cut of size $\kappa(G)$.*

Proof. Just consider a min-cut (in G) (L^*, C^*, R^*) , and consider the intersections $L^* \cap S, R^* \cap S$. Since $|C^*| = \kappa(G) \leq k$, it's impossible for both of them to be empty (using the proof of Lemma 4.4). If one of them is empty, then this is a case for Lemma 4.4. If neither is empty, then this is a case for Lemma 4.3. $\square$

4.1.4 The algorithm is correct

Lemma 4.6. *The algorithm is correct.*

Proof. If $\kappa(G) \leq k$, then by Lemma 4.5, the algorithm returns a cut of size $\kappa(G)$.

On the other hand, if the algorithm returns a cut (L, C, R) , then $|C| \leq k$. By Lemma 4.2, $\kappa(G) \leq |C|$. Since this implies $\kappa(G) \leq k$, it follows (by the above) $|C| = \kappa(G)$. $\square$

4.2 Insight and discussion

When written down, the algorithm looks incredibly simple. Test two kinds of cuts, take the smallest of them, it's the answer if it's within the promise. It's also easy to analyze the complexity of the algorithm.

The difficult parts are to derive the algorithm in the first place, and to prove it is correct.

I will actually discuss in reverse order. First, given the algorithm, I describe how you can prove it's correct. In turn, I'll motivate how the algorithm might come to being, whether given G' or not.

4.2.1 Proving the correctness

Given the algorithm, how do you prove its correctness? Just prove both directions. If the algorithm returns a cut, then it is actually a cut in G . And if the min-cut has size $\kappa(G) \leq k$, then the algorithm will return a cut of size $\kappa(G)$. (Not necessarily the same cut, because there may be different min-cuts.) Therefore, since the algorithm only ever returns cuts of size $\leq k$, if the algorithm returns any cut (L, C, R) at all, it follows $\kappa(G) \leq |C| \leq k$; in turn, this means the algorithm finds a min-cut.

Note: It's not enough to show only one direction. Several students miss proving that, if the algorithm returns a cut, it is a cut in G . This is not obvious, because the algorithm checks for cuts in G' as well. If you don't prove this claim, it's possible to argue, the algorithm found some cut, but it's a cut in G' , it doesn't necessarily mean it's a cut in G .

For these two directions, the proof can further be broken down on what kind of cut it is: one where S appears on both sides, or only on one side. The assumption $|S| = k + 1$ comes in to say, it's impossible that $S \subseteq C$, so these two cases are enough.

4.2.2 Deriving the algorithm given G'

How do we come up with the algorithm in the first place? The last paragraph of the previous discussion leads to this. If we know $\kappa(G) \leq k$, then the min-cut won't cover S entirely. That means S appears on at least one side of the cut. So instead of checking every possible pair of vertices in G , it's enough to check pairs of vertices that include something in S . There are two cases: either both vertices are in S , or only one of them are in S . This corresponds directly to the two steps of the algorithm.

To optimize this further, consider the case where only one vertex is in S . Suppose we have a cut (L, C, R) where $R \cap S = \emptyset$, and let $y \in R$. Now, if we take *any* vertex x in $S \cap L$, the cut (L, C, R) works as an (x, y) -cut. That means, instead of testing all the possible S against y , we can just join all of them into a single vertex r , and test r against y . This is effectively what G' is doing.

As a note, this idea actually works for any k , not just $k = \sqrt{n}$. But the idea of using r only works if S is not split by the cut. If S is split by the cut, we have to test all $O(k^2)$ pairs. This gives rise to a total of $O(n + k^2)$ pairs of oracle queries to do, in which $k = \Theta(\sqrt{n})$ gives the best gain.

4.2.3 Getting the idea of r

If we're not given G' , how would we derive the idea of r ? This is where I'm not entirely sure myself. I mean, a lot of math is having the creative insight to do novel things, such as introducing r here.

But I think it can be motivated. The idea is, if S is not split, then connecting everything in S won't change the cut either. So we can turn S into a clique. The problem is, even in this case, we still don't know which vertices of S will fall into the cut C or not; we cannot query using a vertex in C . So this motivates a new vertex r . Since the goal is to make S connected together, we just say r is adjacent to everything in S . This gives rise to the graph G' , and we find out our motivation can be turned into a proof.

4.3 Grading criteria and common mistakes

4.3.1 Minor error: Not proving the algorithm will not hallucinate

Most students proved that, if $\kappa(G) \leq k$, then the algorithm will return a cut. But there are quite a lot of students that missed the other direction: if the algorithm returns a cut, then $\kappa(G) \leq k$.

This is obvious for cuts in step 2, because those cuts are in G . If the algorithm returns a cut (L, C, R) , then clearly $\kappa(G) \leq |C|$, and $|C| \leq k$ by the construction of the algorithm.

But it's not obvious for cuts in step 1, because those cuts are in G' . You can only say $\kappa(G') \leq k$, but then what? What's the relationship between G' and G ? If you don't explain further, you lose a mark here.